AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH (AIUB)
Faculty of Science and Information Technology
Department of Computer Science
Undergraduate Program

COURSE OUTLINE

I - Course Code and Title: CSC 1102 & 1103: Introduction to Programming

II - Credit: 3/1 credit hours (3 hours of Theory & 3 hours of Lab per week)

III - Nature: Major Course for BSc. in CS

IV- Prerequisite: Nil

V - Course Description:

The goal of this course is to teach the key programming and problem solving skills where the students will be able to design, implement, debug, and test C and C++ programs. This course teaches the fundamentals of C and C++ programming language including – Data types, Operators & Expressions; Standard input/output (I/O); Conditional statements (if…else), Switch; Loops (do...while, while, for); Arrays (single, multidimensional); Pointers, Functions; Structure, Unions & Enumerations; Strings and The Object Oriented Programming (OOP) Principles such as Classes and Objects; Constructors; Operator Overloading; Inheritance and Polymorphism; Encapsulation, Abstract Class, and Interface; C++ Stream Input/Output, File Processing, Templates, Exception Handling, String Processing, Standard Template Library (STL) etc.

VI – Objectives:

At the end of the course, the students will be able to:

1. Understand the role of designing and the development of programming solutions to solve different computational problems.
2. Define aspects of C and C++ programming language.
3. Develop programs and demonstrate programming skills in C and C++ (Structured and Object-oriented Programming).
4. Make good use of compilers, debuggers, and related tools.
5. Develop the pre-requisite programing skills for Data Structure & Algorithm courses.
6. Apply gained knowledge and Skills to the IT industry.

VII – Topics to be covered

Time Frame	Lecture	TOPICS	Specific Objective(s)	Suggested Activities	Teaching Strategy(s)
Week 1	*L1*	• Mission & Vision of AIUB • Starting Concept of Computer Programming • Introduction to C Programming Language.	• Mission & Vision of AIUB • To understand how human being interacts with computer and machines • How computer languages perform as the gateway to bridge up human and computer • Developing simple programs, compiling, and running • C Header files • Getting familiar with C programming environments (IDE's) to write, compile and running C programs	Lecture, Lab: Exercise, **Assignment**	Lecture notes, question-answer session.
	L2	• Variables • Data types	a) Understand the concept of variables in C language. b) Explain different data types available in C language c) Using different types of variables to store data		
Week 2	*L3*	• Operators • Expressions • Type Casting	a) Introduce different types of operators. b) Demonstrate the use of operators. c) Explain expression in C d) Illustrate type casting in C	Lecture, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.
	L4	• Conditional Operators • Flow Chart • Pseudo code	a) Understand the concept of conditional operator and its significance. b) Demonstrate the use of conditional operators c) Solving simple problems and creating flow chart. d) Apply knowledge to develop programs from Pseudo code		

Week 3	*L5*	• If-else statement • switch statement	a) How to branch program control flow using conditional statements? b) To introduce the control structures of if-else and switch c) Using conditional operators through if, switch statement d) Introducing break statement to break a condition	Lecture, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.
	L6	• Loops • while, do-while, for	a) How to perform iterative jobs using loops (while, do-while, for) b) How to repeat the execution of a block of code for a specific number of times c) How to use conditional statements with loops d) How to minimize the size of codes by using loops? e) Infinite loops f) How to convert a loop? (for example, writing a loop written in while to convert in for or do while)		
Week 4	*L7*	• Array • Array Declaration and Initialization • Storing and Accessing Array Elements • Multi-dimensional Arrays	a) How to declare, initialize, store array elements? b) Apply array to reduce declaring too many variables. c) Importance of array for storing and loop through similar type of data. d) Learn when Multi-Dimensional Arrays are useful.	Lecture, **Quiz - 1**, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.
	L8	• Character Array, Strings • String Handling Library Functions	a) What are Strings? Relate Strings and Characters. b) Explain how Strings can be represented using array of characters.		
Week 5	*L9*	• Pointers • Dynamic Memory Allocation	a) Introducing pointers to the students. b) To let students, understand low-level data handling. Let	Lecture, LAB: Exercise, **Assignment**	Lecture notes, question-

			students handle direct memory locations. c) How pointers points to a value of a variable? d) When should we use pointers? e) What are the advantages of using pointers? f) How to access pointers pointing to a variable and the address of a pointer itself?		answer session.
	L10	• Functions • Function call • Passing parameter/s • Returning value • Parameter pass by value, pass by reference • Scope	a) What is a function? b) How to declare, write and call a function? c) How to send arguments (actual parameters) and return values by calling a function? d) Compare different Scope of variables in Functions. e) Apply call by value and call by reference in functions. f) Understand that functions are a great way of reusing block of code and thus minimizes code in a program.		
Week 6	*L11*	• Structure • Structure Members • Structure as Argument • Nested Structure • Union & Enumeration	a) What is a Structure? How to define structure and access structure members? b) Why should we use Structure? c) Compare structure and nested structure. d) How to pass structure as an argument to a function? e) Usage of Unions and Enumerations	Lecture, **Quiz - 2**, **Review for Midterm** Exam, LAB: Exercise, **Assignment**	Lecture notes, question-answer session. **Lab Mid Project Demonstration**
	L12	• File • Basic File I/O • File Access Modes	a) What is a File? How to create, open, read and write data to a file? b) Compare opening file in different modes. c) Explain storing and retrieving data to and from a file.		

			d) File manipulation using pointers.		
Midterm Week Week 7					
Week 8	*L13*	• Introduction to C++ and Object Oriented Programming Principles	a) Introducing C++ and Object Oriented Programming Principles to students b) History of C++ c) Procedural programming vs. object oriented programming (OOP) d) Pillars of OOP development (Class, Objects, Inheritance, Encapsulation, Abstraction and Polymorphism) e) Coding environment (code blocks), Writing first code in C++, Basic syntax of C++ programming (input, output, header file, etc.) f) Basic building blocks of C++ program (namespace, scope resolution operator, syntax for input, output, header file, etc.)	Lecture, LAB: Exercise, **Assignment**	Lecture, Student reporting and Board work
	L14	• Classes and Objects in C++ • Properties, Functions • Access Modifiers	a) What are Classes and Objects in Object-Oriented programming? b) Real-life example of conceptualizing classes and objects c) Writing Classes with properties and functions d) Access modifiers (private, public, protected) and their purpose. e) Application of access modifiers for class, and class members (variables and function). f) Accessing class data member and member function from inside and outside of the class. g) Memory mapping for Class, object and their members.		

Week 9	*L15*	• Functions and Operators in C++ • Setter and getter functions • Operator Overloading	a) Declaring and defining functions. b) Setter, getter functions in a Class. c) Passing arguments to a function (call-by-value, call-by-reference). d) Default arguments /parameter for functions. e) Operators in C++ f) Why operator overloading?	Lecture, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.
	L16	• Constructor and Destructor • Static class member in C++	a) Constructor and Destructor: what and why? b) Default constructor, Parameterized constructor. c) Constructor and destructor calling order. d) Static class member		
Week 10	*L17*	• Inheritance • Multiple Inheritance	a) Base class and derived class in relation to inheritance (real life example) b) What is inherited in inheritance /Access control in Inheritance (public, private and protected) c) Single, Multiple, and multi-level inheritance	Lecture, **Quiz - 3**, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.
	L18	• Abstraction and Polymorphism	a) Data Abstraction and benefits of data abstraction, Interface in data abstraction. b) Virtual function, object pointers, dynamic linkage, late binding c) Polymorphism, Pure virtual function d) Function overloading (polymorphism)		
Week 11	*L19*	• Stream Input/Output • File Processing	a) Read from a file, writing to a file b) Standard C++ library for reading and writing from/to a file, opening and closing a file. c) Compare C and C++ file manipulation mechanisms	Lecture, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.

	L20	• String in C++	a) String representation in C++ (C-style character string, String class) b) Sting manipulation in C++ (Assignments and concatenations, copying, comparing, finding, etc.)		
Week 12	L*21*	• Exception Handling • C++ Standard Exceptions • User-defined Exceptions	a) What are Exceptions? How to handle Exceptions? b) Try and catch block in exception handling, throwing an exception, catching an exception, c) default and multiple catch blocks, nested try blocks d) How to utilize C++ standard Exceptions? e) How to create user-defined exception handler?	Lecture, **Quiz - 4**, LAB: Exercise, **Assignment**	Lecture notes, question-answer session.
	L22	• C++ Namespaces	a) What are Namespaces in C++? b) Why should we use Namespaces? c) How to use Namespaces?		
Week 13	*L23*	• Templates • Standard template Library in C++	a) Function templates, class templates, b) Sequence Containers (vector, list, deque), Associate Containers (multiset, set, multimap, map) c) Container Adapters (stack, queue, priority queue)	Lecture, **Review for Final Exam** LAB: Exercise, **Assignment**	Lecture notes, question-answer session. **Lab Final Project Demonstration**
	L24	• Preprocessor Directives	a) What are Preprocessors in C++? b) What are the purposes of Preprocessors? c) How to use Preprocessors?		
Final Exam Week **Week 14**					
Lab Exam **Week 15 (Exact Date will be Announced Later)**					

VIII- Course Policies:

1. **Student Attendance**
   All students are expected to attend all scheduled classes, and to read all assigned chapters / materials before coming to class. Attendance will be taken at the beginning of each class period.

2. **Class Participation & Peer Evaluation**
   You Students are expected to participate actively in the class. Your contribution towards your team will be counted too. Performance will be recorded during the theory lectures and labs. You must carry the text book with you during the class and lab. If you do not bring the book, then 0 will be given in the performance.

3. **Quiz & Exam**
   For both terms, there will be 2 quizzes, better quiz will be counted for grading. In other word, there will be four quizzes and better two quizzes will be counted for the course. There will be Lab exam in addition to Lab Performances based on Lab Task / Assignment given by the instructor. The details will be provided during Labs. During the quiz and exam, if you practice any unfair means, I will give you UW or F for the rest of the semester.

   No Make-up Quiz will be taken!!!

4. **After Grade submission:**
   Once you complete the exam, your grade will be published on VUES. If you are not satisfied with your grade or if you think your marks will increase, then you can come for negotiation. In that case, I will remove any bonus mark given to you previously and we will negotiate. There is a high chance that you may receive a lower grade after the negotiation. So, be careful what you wish.

5. **Poor grades in Midterm Exam:**
   Students with “F” or “I” in the midterm, must drop the course right after the mid-term exam. If you wish to continue with “F” at your own risk and if you receive “F” in the final, you won’t be allowed to drop the course. You must drop the theory and Lab both.

IX– Evaluation

Mid	**Theory**	**(3 credit)**	**Contribution**
	Attendance:	10%	**50 %**
	Performance:	10%	
	Quiz:	30%	
	Mid-term Exam:	50%	
Final	Attendance:	10%	**50%**
	Performance:	10%	
	Quiz:	30%	
	Final Exam:	50%	
	Total:		**100%**

Mid	**Lab**	**(1 credit)**	**40 %**
	Attendance:	10%	
	Lab Performance:	5 x 10%= 50%	
	Project (Part-1 in C):	40% (Deadline: 6th week)	
Final	Attendance:	10%	**40%**
	Lab Performance:	5 x 10%= 50%	
	Project (Part-2 in C++):	40% (Deadline: 13th week)	
	Lab Exam	(TBA: 15th Week)	**20%**
	Total:		**100%**

X – Textbook/ References

1. Let Us C, 13th Edition, by, Yashavant Kanetkar.
2. Teach Yourself C, 3rd Edition, by Herbert Schildt.
3. C++ How To Program, 8th Edition, by Deitel and Deitel.
4. C: The Complete Reference, 4th Edition, by Herbert Schildt.
5. Teach Yourself C++, 3rd Edition, by Herbert Schildt.
6. The C++ Complete Reference, 4th Edition, Herbert Schildt.
7. The C++ Programming Language, Special 3rd Edition, Bjarne Stroustrup
8. Thinking in C++, Volume One, 2nd Edition. Bruce Eckel.
   Downloadable from http://www.BruceEckel.com.
9. The C++ Programming Language by Bjarne Stroustrup
10. C++ Primer Plus, 5th Edition, by Stephen Prata
11. Essential C++ (The C++ In-depth Series), by Stanley B. Lippman
12. C and C++ reference, url www.cplusplus.com/
13. C and C++ reference, url www.cprogramming.com/reference/
14. The Lab Manual of PL1 and PL2